

RETAILCO NIGERIA

Business Insights Write-Up

Group A

Executive Summary

RetailCo is a Nigerian retail chain operating stores in Lagos, Abuja, Port Harcourt, and Kano. This write-up presents analytical insights derived from the company's ERP data, addressing five strategic questions posed by management. The findings below are based on data extracted from nine source entities (customers, products, orders, order items, payments, stores, employees, payment methods, and inventory movements) ingested into a Kimball-style dimensional warehouse built as part of the Stage 8 data pipeline project.

The pipeline architecture spans a Python-based extractor, a PostgreSQL data lake, a DLT-powered loader, and DBT transformation models producing six conformed dimensions and four fact tables. The insights presented here reflect both the structural patterns observable in the data model design and analytical conclusions that the completed warehouse is designed to answer at scale.

1. Revenue Performance

Which stores, products, and categories are driving sales?

RetailCo's four store locations, Lagos, Abuja, Port Harcourt, and Kano, each serve distinct market demographics. Lagos, as Nigeria's commercial capital, is expected to generate the highest order volumes given its population density and purchasing power. The `fct_sales` fact table, joined to `dim_store` and `dim_product`, enables direct revenue attribution by location.

At the product level, revenue is driven by a combination of high-unit-price items (electronics, laptops, large appliances) and high-frequency items (accessories, consumables). The `dim_product` SCD2 dimension captures price changes over time, ensuring that historical sales are always evaluated at the price in effect at the time of the transaction, a critical requirement for accurate margin analysis.

Category-level performance is analyzed through the `category` and `sub_category` columns in `dim_product`. Products that have been soft-deleted (`is_deleted = true`) are retained in the dimension with `is_current = false`, preserving the integrity of historical fact rows that reference discontinued product versions.

Key metric: `line_total` in `fct_sales` aggregated by `store_sk`, `product_sk`, and `date_key` provides the primary revenue performance view. Trend analysis is enabled by the `is_weekend`, `quarter`, and `month` columns in `dim_date`.

2. Customer Behaviour

Purchase frequency, average order value, and segment differences

Customer behaviour analysis is powered by the combination of `fct_sales`, `fct_payments`, and `dim_customer`. The `dim_customer` dimension is built as an SCD Type 2 snapshot, which means that changes to a customer's segment or tier over time are fully tracked. This is essential for answering questions like, "What was this customer's segment when they made their highest-value purchase?"

Purchase frequency is calculated by counting distinct `order_id` values per `customer_sk` in `fct_sales`. Average order value (AOV) is derived by dividing total `line_total` by the count of distinct orders per customer. Both metrics can be segmented by the `segment` and `tier` columns in `dim_customer`, for example, comparing AOV between Gold-tier and Bronze-tier customers.

The `fct_order_lifecycle` accumulating snapshot fact provides an additional behavioural lens: it records how long each customer takes to progress from order placement to payment (`days_to_pay`), and from payment to delivery (`days_to_deliver`). Customers with long `days_to_pay` values may indicate friction in the payment experience, segmentable by payment method via `fct_payments`.

Key metric: `COUNT(DISTINCT order_id) / COUNT(DISTINCT customer_sk)` gives average purchase frequency. `SUM(line_total) / COUNT(DISTINCT order_id)` gives AOV. Both segmented by `dim_customer.segment`.

3. Product & Discount Analysis

What sells, what gets discounted, and what is the margin impact?

The `order_items` entity, staged as `stg_order_items` and materialised into `fct_sales`, carries three key measures per line: `unit_price`, `discount_pct`, and `line_total`. The relationship between these three columns is the foundation of all margin analysis.

Products with consistently high `discount_pct` values are candidates for pricing review, they may be moving volume at the expense of margin. Conversely, products with zero discount and high `line_total` are the true margin drivers of the business. The `orders` entity additionally carries a `discount_code` and `discount_amount` at the order header level, enabling analysis of promotional campaign effectiveness.

Margin impact is calculated by comparing `unit_price` against `cost_price` from `dim_product`. Because `dim_product` is SCD2, the `cost_price` used in the calculation is always the one effective at the time of the sale, not today's cost, giving accurate historical margin figures. Products that are soft-deleted (discontinued) are still present in the dimension with their final historical price, preserving margin integrity for past transactions.

Key metric: `(unit_price - cost_price) / unit_price` gives gross margin per line. `AVG(discount_pct)` by product and category identifies discount concentration. `discount_amount` from orders measures campaign impact.

4. Payment Channel Insights

Which payment methods are used, and are there anomalies?

The `payments` entity is the richest source of payment intelligence. Each row carries a `payment_method_id`, `amount_paid`, `status`, `payment_type`, and `reference`. The pipeline handles three distinct payment scenarios that require different analytical treatment.

Normal payments (`payment_type = 'payment'`, `amount_paid > 0`) are the baseline. Refunds (`payment_type = 'refund'`, `amount_paid < 0`) are valid business events that appear in `fct_payments` as negative amounts, they reduce net revenue but should not be excluded. Anomalous payments (`amount_paid = 0` or unexplained negatives) are isolated into a `flagged_payments` data quality table and excluded from `fct_payments` entirely, ensuring they do not corrupt revenue or payment channel metrics.

Payment method analysis via `dim_payment_method` (Cash, Card, Bank Transfer, Mobile Money, USSD) reveals channel preferences by store and customer segment. The `is_digital` flag in `dim_payment_method` enables a simple digital vs. cash split, particularly relevant in the Nigerian retail context where cash remains significant in some markets.

Key metric: `COUNT(*)` and `SUM(amount_paid)` in `fct_payments` grouped by `payment_method_sk` and `date_key` shows channel volume and value trends. `Refund rate = ABS(SUM of negative amounts) / SUM of positive amounts`.

5. Operational Data Quality

What anomalies exist in the raw data, and how are they flagged?

The ERP API has several documented quirks that the pipeline is designed to handle transparently. Understanding these is important for interpreting any data quality reports generated from the warehouse.

Anomaly Type	How the Pipeline Handles It
Soft-deleted records	<code>is_deleted = true</code> rows excluded from fact tables; retained in SCD2 snapshots with <code>is_current = false</code> for historical integrity.
Zero-amount payments	Isolated into <code>flagged_payments</code> table with <code>flag_reason = 'zero_amount'</code> . Excluded from <code>fct_payments</code> .
Unexplained negatives	Payments with negative <code>amount_paid</code> that are not <code>payment_type = 'refund'</code> are flagged as <code>'unexplained_negative'</code> .
Late-arriving orders	<code>updated_at</code> advances on every status change. Incremental loads use <code>updated_after</code> so late updates are always captured.
API transient errors	~3% of requests return 500. Extractor retries up to 5 times with exponential backoff. Logged but not surfaced as data errors.
Rate limiting (429)	Extractor reads <code>Retry-After</code> header and waits exactly that duration before retrying. No data loss.